```
7h Domain - GraphRAG & Knowledge Graphs
You are the Learner Gem. An expert, coach, and you have great teaching skills & in-
-depth knowledge to help me achieve in-depth level knowledge for this
particular domain. Teach me this end-to-end with proper explanation,
examples, illustrations, etc. Make it super comprehensive. Below are the
topics that you provided under this domain
# GraphRAG & Knowledge Graphs
## Knowledge Graph Basics
```

Show more

Let's go deep on GraphRAG and Knowledge Graphs. This is one of the most differentiated domains in RAG interviews — most candidates know vectors, few understand graphs.

Why Graphs Beat Vectors for Certain Questions

Before diving into the technical details, let me crystallize the core insight. A vector database answers the question: *"which chunks are semantically similar to this query?"* That's a proximity question in embedding space.

A knowledge graph answers: *"how are these entities related, and what can I infer by traversing those relationships?"* That's a structural reasoning question in a semantic network.

The failure mode vectors can't solve: *"Who are all the people who worked at companies that were acquired by fund_X?"* This question requires traversing: Person -> [worked_at] -> Company -> [acquired_by] -> fund_X. No single chunk contains this answer. Vector similarity returns chunks mentioning these entities but can't follow the relational chain. A graph traverses it directly.

1. Knowledge Graph Fundamentals

A knowledge graph represents the world as a network of entities connected by typed relationships. The atomic unit is the **triple**: (subject, predicate, object) — also called a fact.

From an insurance document:

- (fund_A5 Plan A, has_deductible, $500)
- (fund_A5 Plan A, covers, Dental Services)
- (Dental Services, includes, Teeth Cleaning)
- (Gallagher, distributes, fund_A5 Plan A)
- (Marsh5k Plan B, compares_with, fund_A5 Plan A)

Every piece of knowledge expressed as subject predicate object. Stack thousands of these and you get a graph where querying means graph traversal rather than vector lookup.

Property Graphs vs RDF — The Two Major Standards

Property graphs (Neo4j, ArangoDB, Amazon Neptune) prioritize developer ergonomics: pragmatic approach. Nodes and edges can both have arbitrary properties as key-value pairs. A `Person` node might have `{name: "Ankit", role: "Data Scientist", company: "Gallagher"}`. The query language is Cypher (Neo4j) or Gremlin (Apache). Excellent for application development. Native database storage with ACID transactions.

RDF (Resource Description Framework) is the semantic web standard. Everything is a triple, including properties of properties. Supports reification (statements about statements). Underlying data model is simpler, but modeling complex relationships can be verbose. Query language is SPARQL. More formal but less flexible for rapid development.

For RAG applications, property graphs win overwhelmingly: Neo4j is far easier to work with programmatically, LangChain has first-class Neo4j support, and the flexible property model maps naturally to extracted entity attributes.

Graph Databases The Production Options

Neo4j is the undisputed leader for property graphs. Native graph storage (not a relational DB with graph layer). Cypher query language is declarative and readable. Mature ecosystem, now built in (Neo4j 5.x). Enterprise clustering, Python driver + LangChain integration. The production choice for GraphRAG.

ArangoDB is multi-model: combines property graph, document store, and key-value. AQL query language. Good for when you need graph + document queries in one system. Less ecosystem support than Neo4j for RAG.

Amazon Neptune is AWS-managed, supports both property graph (Gremlin) and RDF (SPARQL). Managed service overhead. Tightly integrates with AWS ecosystem. The choice for AWS-native architectures.

Graph Embeddings

You can embed graph structure just like text. Graph-embedding models (Node2Vec, TransE, RotatE, BERT-based) learn dense vector representations of entities and relations such that the geometric relationship in embedding space mirrors the structural relationships in the graph. `Transe` models the relationship `r` between head entity `h` and tail entity `t` as a translation: `h + r ≈ t`. So if `Sunlife Plan A + has_deductible ≈ $500`, then querying with the vector `Sunlife Plan A + has_deductible` should land near `$500` in embedding space.

In a GraphRAG context, graph embeddings allow you to run vector similarity search over the graph structure itself — not just document text.

2. KG Construction from Text — The Engineering Challenge

Building a knowledge graph from unstructured text is the hard part. Here's the full pipeline:

1 Schema definition
Define entity types and allowed relations BEFORE extraction. For insurance: entity types = [Insurance Plan, CoverageCategory, MonetaryValue, Carrier, Broker]. Relation types = [covers, has_deductible, excludes, offered_by, compares_with]. A schema constrains the LLM extractor and prevents noise. Without schema, you get thousands of idiosyncratic relation types that can't be consolidated.

2 LLM-based entity extraction
Prompt LLM to extract entities from each chunk, constrained by your schema. Modern approach: use structured output / function calling to guarantee JSON schema compliance. GLiNER (open-source) is a lightweight alternative for NER without LLM overhead — supports custom entity types with few examples. Spacy + custom NER for high-throughput pipelines.

```
{"entities": [{"name": "Plan A", "type": "InsurancePlan"},
{"name": "$500", "type": "MonetaryValue"}, "relations": [...]}
```

3 Coreference resolution — the critical preprocessing step
Resolve pronouns and aliases before extraction. "It covers dental" → "Plan A" — either to the same entity. Without coreference resolution, you create duplicate nodes. Fuzzy/LLM matching at scale (expensive) resolves pronouns to canonical entity mentions within a chunk. Cross-document LLM-based entity disambiguation: "Sun Life Plan A" vs plain "Plan A" in doc 2 are the same entity — string normalization + fuzzy matching resolves this.

4 Relation classification
For each pair of extracted entities in a chunk, classify the relationship. Joint prompt: "Given entities [Plan A,$500] extract relations", one or more relationship from the list [covers, has_deductible, excludes, ...]". Rebel, and REBEL (open-source relation extraction model) work without LLM for high-throughput. Always extract with confidence scores — low confidence relations introduce garbage edges.

5 Ontology alignment + graph ingestion
Map extracted relations to your ontology (Neo4j schema). Merge duplicate entities across entity, different surface forms). Set confidence thresholds — only ingest triples above 0.8 confidence. Write to Neo4j via MERGE (upsert semantics). `MERGE (p:Plan {name: "Plan A"}) MERGE (d:Deductible {amount: "$500"}) MERGE (p)-[:HAS_DEDUCTIBLE]->(d)`. MERGE prevents duplicate nodes on re-ingestion.

3. Microsoft GraphRAG — Architecture Deep Dive

GraphRAG is Microsoft Research's 2024 system that combines knowledge graphs with LLM-generated summaries to answer questions that require reasoning over entire corpora. It's particularly powerful for "global" questions — ones that require synthesizing information across many documents rather than retrieving a specific fact.

The key innovation over plain RG RAG is the hierarchical community structure: instead of just having a graph of entities, GraphRAG detects communities of closely related entities and generates natural-language summaries of each community. These summaries become the retrieval units for global queries.

The Leiden Algorithm — Why This Specific Algorithm

GraphRAG uses Leiden (Traag et al., 2019) rather than the older Louvain algorithm. Both detect communities by maximizing modularity — a measure of how much more densely connected a community is internally versus to the rest of the graph. Leiden fixes a critical Louvain bug: Louvain can produce internally disconnected communities (nodes in the same "community" that aren't actually reachable from each other). Leiden guarantees internally connected communities.

The algorithm:

1. Start: each node is its own community
2. Locally move nodes: if moving a node to an adjacent community increases modularity, do it
3. Aggregate: merge nodes within a community into a super-node
4. Repeat on the aggregated graph
5. Stop when no moves improve modularity

The `resolution_parameter` controls granularity: higher γ = more, smaller communities; lower γ = fewer, larger communities. GraphRAG generates communities at multiple γ values to create the hierarchy.

Global vs Local Search — The Two Query Modes

Local search handles questions about specific entities: "What does Plan A cover for dental?" The path: identify the entity ("Plan A") → traverse the graph (`[Plan A] -[covers]-> [CoverageType: Dental]`) → retrieve the linked source chunks → generate answer. Fast, precise, grounded in specific facts.

Global search handles questions about themes across the entire corpus: "What are the main differences between all the insurance plans in our documents?" The path: embed the query → search community summaries at each hierarchy level → select the most relevant communities → map-reduce synthesis (each community summary generates a partial answer, a final LLM call reduces all partial answers into one). This is the killer feature — no naive RAG system can answer global theme questions well.

Map-Reduce synthesis in detail

1. Map step: for each relevant community summary, prompt the LLM: "Given this community report, what's the relevant information for query Q?" Generate a partial answer for each community.
2. Reduce step: collect all partial answers, prompt the LLM: "Synthesize these partial answers into a comprehensive response."

This scales to arbitrarily large corpora — you never try to fit the whole corpus in one context window.

4. LightRAG — Simpler, Faster, Incrementally Updatable

LightRAG (Guo et al., 2024) is a leaner alternative to GraphRAG that addresses one of GraphRAG's biggest limitations: slow indexing. GraphRAG's community detection and summary generation is expensive and requires rebuilding from scratch when documents are added. LightRAG supports incremental updates.

LightRAG's dual-level retrieval works as follows: for "low-level" specific facts about named entities, it searches the entity graph directly. For "high-level" queries (broader concepts and themes), it searches concept-level graph summaries. The hybrid mode combines both with vector search, fusing results via RRF. Entity deduplication is LightRAG's weakness: when a new document is ingested, extracted entities are compared against the existing graph using fuzzy string matching + embedding similarity. Entities with similarity above a threshold are merged rather than duplicated.

5. Neo4j + RAG — The Production Integration

Neo4j is the go-to graph database for RAG applications. The key reasons: Cypher is the most readable graph query language, LangChain has a native `Neo4jGraph` and `Neo4jVectorIndex`, and Neo4j 5.x adds native vector indexes alongside the graph.

Pattern 1: Text2Cypher — natural language to graph query

Provide the LLM with your Neo4j schema. Ask it to generate a Cypher query that answers the natural language question. Execute the query against Neo4j. Pass results to LLM for final response generation.

```
# Schema: (:InsurancePlan) -[:COVERS]-> (:CoverageCategory)
# (:InsurancePlan) -[:HAS_DEDUCTIBLE]-> (:Money)

from langchain_community.graphs import Neo4jGraph
from langchain_core.prompts import GraphCypherQAChain

graph = Neo4jGraph(url="bolt://localhost:7687")
chain = GraphCypherQAChain.from_llm(llm, graph=graph, verbose=True)
chain.invoke("Which plans cover dental and have deductible under $500?")

# Generated Cypher:
MATCH (p:InsurancePlan) -[:COVERS]-> (c:CoverageType {name: "Dental"})
MATCH (p) -[:HAS_DEDUCTIBLE]-> (d:Money)
WHERE d.value < 500
RETURN p.name, c.name, d.value
```

Pattern 2: Graph vector hybrid search (Neo4j 5.x)

Store embeddings on graph nodes. Run vector similarity search within a graph traversal. Find semantically similar entities AND their graph neighbors in one query — the killer feature of Neo4j vector indexes.

```
# Create vector index on Chunk nodes:
CALL db.index.vector.createNodeIndex(
  'chunk_embeddings', 'Chunk', 'embedding', 1536, 'cosine'
)

# Query: vector search + graph expansion:
CALL db.index.vector.queryNodes('chunk_embeddings', 10, $query_embedding)
YIELD node AS chunk, score
MATCH (chunk) -[:MENTIONS]-> (e:Entity)
WHERE score > 0.75
RETURN chunk.text, score, type(r), related_name, score
```

Pattern 3: Schema guided extraction into Neo4j

Use LangChain's LLMGraphTransformer to extract structured graph data from text using your schema definition, then write directly to Neo4j.

```
from langchain_experimental.graph_transformers import LLMGraphTransformer

transformer = LLMGraphTransformer(
    llm=llm,
    allowed_nodes=["InsurancePlan", "CoverageType", "Money"],
    allowed_relationships=["COVERS", "HAS_DEDUCTIBLE", "EXCLUDES"]
)

graph_docs = transformer.convert_to_graph_documents(documents)
graph.add_graph_documents(graph_docs, include_source=True)
```

The `include_source=True` parameter in `add_graph_documents` is critical for RAG — it creates `[:MENTIONS]` edges from each source document chunk to the entities it mentions. This lets you do hybrid graph vector retrieval: find semantically similar chunks, then expand to their entity neighbors, then retrieve all chunks that mention those entities. You get both semantic similarity AND structural graph context in one query.

Interview Question Bank — Domain 7

Basic **What is a knowledge graph triple and how does it differ from a vector embedding?**

A triple is an atomic fact expressed as (subject, predicate, object) (Plan A", has_deductible", "$500"). It's symbolic, discrete, and human-readable. The relationship is explicit and typed. A vector embedding is a dense numerical representation of a chunk of text. It's continuous, opaque, and captures semantic similarity geometrically, not relationships. Triples store knowledge explicitly, embeddings store knowledge implicitly. A weights. Triples support precise graph traversal and multi-hop reasoning ("find all plans that have the same coverage as Plan X"). Embeddings support fuzzy semantic similarity ("find chunks similar to this query"). Production GraphRAG combines both: the graph provides structural reasoning, embeddings provide semantic retrieval. Neither subsumes the other.

Basic **What is the Leiden algorithm and why does GraphRAG use it?**

Leiden is a community detection algorithm that partitions a graph into densely connected groups of nodes (communities) by maximizing modularity — a measure of how much more connected a community is internally versus externally. GraphRAG uses it to discover thematic clusters of entities in the extracted knowledge graph, then generates LLM summaries of each community. These community summaries become the retrieval units for global queries. GraphRAG specifically uses Leiden rather than the older Louvain algorithm because Leiden guarantees internally connected communities — Louvain can produce communities where some nodes are unreachable from others within the same community, which would create nonsensical summaries. The `resolution_parameter` controls granularity, and GraphRAG runs Leiden recursively at multiple levels to create a hierarchy.

Mid-level **Explain GlobalSearch vs LocalSearch in GraphRAG. When would each be used?**

LocalSearch handles specific entity-centric questions: "What are Plan A's deductibles?" Path: identify entity ("Plan A" → traverse graph neighborhood → retrieve linked source chunks → generate answer. Fast, grounded, uses both graph structure and source text. GlobalSearch handles thematic questions across the entire corpus: "What are the main coverage themes across all our insurance plans?" Path: embed query → search community summaries at hierarchy level → map-reduce synthesis (each community summary generates a partial answer, final LLM call synthesizes partial answers). GlobalSearch is slow and expensive (many LLM calls) but enables questions no single context window can hold the entire corpus — community summaries are the level of abstraction that. Flag: use LocalSearch for 80% of RAG queries (specific facts), Use GlobalSearch when the user explicitly needs a synthesis across the entire knowledge base.

Mid-level **What is co-reference resolution and why is it critical for knowledge graph construction?**

Coreference resolution identifies when different text spans refer to the same real-world entity. In Plan A covers dental. It covers vision. It excludes pre-existing conditions", "It" refers to Plan A. Without resolution, you create "It" and "the plan" both refer to "Plan A". Without resolution this, you'd extract "It" as a separate entity, polluting the graph. The same problem occurs cross-sentence and cross-document: "Sun Life's Plan A." "this Sun Life offering" may refer to the same entity in different documents. Failing to merge these creates duplicate nodes that split the graph's knowledge about that entity across disconnected fragments — queries miss connections that should exist. Approaches: rule-based (spaCy coreferee, AllenNLP coreference resolution), spaCy coreferee, or LLM-based disambiguation. In practice, also apply entity normalization (lowercase, remove punctuation) and fuzzy matching (Levenshtein distance) for non-pronoun entity resolution before graph ingestion.

Hard **Design a GraphRAG system for an insurance company with 50,000 policy documents that update weekly and need both specific fact queries and thematic analysis queries.**

Given the weekly update requirement, full GraphRAG's full rebuild is impractical. I'd use a hybrid architecture: LightRAG for the incrementally updated knowledge graph, augmented with RAPTOR-style community summaries regenerated weekly for global queries. Architecture: (1) Document ingestion: parse PDF → chunk (recursive, 512 tokens, 10% overlap) → extract entities + relations (GPT-4o-mini, few-shot examples) → entity resolution (fuzzy matching + embedding similarity) → upsert to Neo4j. (2) Weekly batch: detect changed documents (hash comparison) → re-extract → incremental graph update → Neo4j MERGE (upsert). (3) Weekly batch: detect changed graph communities (using Leiden + regenerate summaries only for affected communities. (3) Query routing: in LangGraph, classify query as local (specific entity) or global (thematic) → route to LocalSearch or GlobalSearch. (4) Local search: entity linking → 1-hop graph traversal → source retrieval. (5) Global search: map-reduce over community summaries. Cost: ~$50/month for embeddings + ~$200/month for LLM summarization (only changed communities). Latency: local ~2s, global ~8s. Failure modes: (1) Hybrid retrieval: combine vector search + BM25 + graph traversal. (2) Fuse with RRF. Schema: must define entity types (Insurance Plan, Coverage, Exclusion, Amount, Carrier, Regulation) and relations (up-front) — changing schema later requires re-extraction. Graph-enhanced RAG (Microsoft, 2024) reports 26% improvement over naive RAG on multi-hop questions. Cost: ~$1/1K pages for extraction and summarization at indexing time add significant cost for large corpora. Estimate: ~$50-200 per 5000 documents with GPT-4o-mini.

The Master Mental Model for GraphRAG

Vectors answer "what is similar?" — Graphs answer "how are these connected?"

Vector RAG is a library where you find books by searching for similar topics. GraphRAG is a library where you find books AND can follow citations, see which authors collaborated, trace which ideas influenced others — and answer questions that require understanding the network, not just individual documents.

The practical decision rule:

If your users ask specific questions with answers in individual passages → plain vector RAG is sufficient.

If your users ask questions like "what are all the plans that_", "multi-hop research", "summarize how our policies compare across _", "timeseries" (global synthesis), or "what entities are connected to X through what relationships" (graph reasoning) → GraphRAG or

You are out of free messages until 10:00 PM Upgrade
Reply...
Sonnet 4.6
Claude is AI and can make mistakes. Please double-check responses.